

Introducción

Este curso enseña a construir agentes y sistemas RAG (Retrieval-Augmented Generation) utilizando un único framework de orquestación —LangGraph— y modelos de lenguaje ejecutándose localmente mediante Ollama y, en producción, llama.cpp. La elección de un solo stack es deliberada: en 45 horas es más útil dominar bien una herramienta que tocar tres por encima. Y la elección de modelos locales tiene tres ventajas concretas para el aula: coste cero para el alumno, privacidad total de los datos que se procesan, y comprensión real de qué hace una inferencia LLM por dentro.

El curso es eminentemente práctico. Cada concepto se introduce con un fragmento de teoría corto seguido de código que el alumno escribe y ejecuta en su propio equipo. Los nueve días siguen una progresión deliberada: tres días de fundamentos para asentar el ciclo agente-tools-RAG, tres días intermedios para entrar en orquestación con grafos y técnicas avanzadas de retrieval, y tres días finales centrados en lo que separa un prototipo de un sistema utilizable: evaluación, observabilidad, despliegue y trade-offs entre modelos de distintos tamaños.

Perfiles del alumnado

Perfil	Punto de partida	Lo que esperamos al final
A · Principiante	Programación básica, poco o ningún Python	Capaz de modificar agentes existentes, ejecutar pipelines RAG sobre sus propios documentos y depurar problemas comunes
B · Intermedio	Python sólido, ha consumido APIs de LLM antes	Capaz de diseñar y desplegar un sistema de agentes propio en local, con evaluación, observabilidad y un modelo cuantizado en producción

Stack del curso

Componente	Herramienta	Versión / modelo de referencia
Orquestación	LangGraph	langgraph ≥ 0.2
Servidor LLM (desarrollo)	Ollama	0.6.x
Servidor LLM (producción)	llama.cpp / llama-server	build de 2026

Componente	Herramienta	Versión / modelo de referencia
Modelo inicial	Qwen3-8B-Instruct (Q4_K_M)	~5 GB, corre en 16 GB RAM
Modelo final	Qwen3-14B o Qwen3-30B-A3B	8–18 GB, mejor tool use y razonamiento
Embeddings	nomic-embed-text vía Ollama	768 dim, multilingüe decente
Vectorstore	Chroma	persistencia local, sin servicios externos
API	FastAPI	0.115+
Despliegue	Docker + Railway	Hobby plan (~5–15 €/mes)

Por qué Ollama y luego llama.cpp

Ollama oculta la complejidad: un comando, un modelo descargado, una API compatible con OpenAI ya escuchando. Perfecto para los siete primeros días.

llama.cpp con llama-server da control fino sobre cuantización, número de capas en GPU, batch size y context window. Se introduce en el día 8 cuando hablamos de optimización y costes reales.

Ambos exponen un endpoint OpenAI-compatible, así que el código del agente apenas cambia entre desarrollo y producción.

Métricas de éxito del curso

- El alumno completa el proyecto final ejecutándose 100% en local, sin claves API externas.
- El sistema final responde con groundedness verificable y faithfulness $\geq 0,8$ en el eval set propio.
- El alumno entiende qué cuantización elegir según su hardware y sabe medir el trade-off calidad / velocidad.
- El alumno sabe leer y modificar un StateGraph de LangGraph con confianza.
- El alumno tiene un Dockerfile funcional y conoce el flujo de despliegue en Railway.
- El alumno sabe diagnosticar los tres fallos típicos: contexto desbordado, herramienta mal llamada por el modelo, y retrieval que devuelve chunks irrelevantes.
- El alumno se va con un sistema observable, no con una caja negra.

Mapa del curso

El curso se estructura en tres fases de tres días cada una. Cada fase amplía el sistema construido en la anterior: los días 1–3 dejan funcionando un agente con tools y un RAG mínimo, los días 4–6 lo convierten en un sistema orquestado con retrieval de calidad, y los días 7–9 lo preparan para que otra persona pueda usarlo sin el instructor presente.

Fase 1 · Fundamentos (días 1–3)

- Día 1 — Setup completo, primer modelo local, primera llamada, primer agente con una tool.
- Día 2 — Tool use serio: clase Agent, múltiples tools, validación, errores.
- Día 3 — RAG básico end-to-end: embeddings locales, FAISS, primer pipeline funcional.

Fase 2 · Intermedio (días 4–6)

- Día 4 — Chunking, Chroma con persistencia, problemas reales de un vectorstore en uso.
- Día 5 — LangGraph en profundidad: StateGraph, memoria, ReAct, planner.
- Día 6 — Hybrid search, reranking, evaluación con métricas y LLM-as-judge local.

Fase 3 · Producción (días 7–9)

- Día 7 — Sistemas multi-agente con LangGraph: supervisor, debate, editor-loop.
- Día 8 — Observabilidad, testing, paso de Ollama a llama.cpp, cuantización y elección de modelo.
- Día 9 — FastAPI, Docker, despliegue en Railway, demo final.

Distribución temporal por temática

Fundamentos de LLM y agentes: 20% (días 1–2)

RAG (todas sus fases): 35% (días 3, 4, 6 y como componente en proyectos)

LangGraph y orquestación: 25% (días 5, 7)

Producción, evaluación y despliegue: 20% (días 8, 9)

